SolariNet migration runbook

Candidate procedure · 20 October 2026 · monitoring DB and SoR

> **Production gate:** Current live databases have no `solari_migration_ledger`. The runner intentionally refuses them. Do not stamp their history from table presence alone; audit a backup and compare the actual schema against every historical migration first.

1. What the runner does

`tools/migrate.py` applies full file IDs in order, including both distinct `016_*` monitoring migrations. It stores each SHA-256 checksum and an `applying` / `applied` state in the target database. It refuses edited or missing historical files, gaps, interrupted runs, and nonempty databases with no ledger. MariaDB DDL is not transactional; an interrupted file requires inspection rather than automatic replay.

Stream	Initial input	Files proven on scratch DB
Monitoring	`db/migrations/001_baseline.sql` through `020_ctl_request_queue.sql`	21 applied, including both 016 files
SoR	`netdb/sor/schema.sql`, then its numbered migrations	6 applied through 021

2. Clean installation

Create a new empty database with an account permitted to apply DDL. Supply credentials through a protected MariaDB option file, a local Unix socket account, or a process environment managed outside git. Run from any directory; the script resolves SQL paths from its own location.

```
python3 tools/migrate.py --database monitoring --db-name NEW_MONITORING_DB --check
python3 tools/migrate.py --database monitoring --db-name NEW_MONITORING_DB
python3 tools/migrate.py --database monitoring --db-name NEW_MONITORING_DB --check

python3 tools/migrate.py --database sor --db-name NEW_SOR_DB --defaults-file /path/
to/protected.cnf
```

The final `--check` should produce no `PENDING` lines. The deprecated `solariServer --init-db` now exits with code 2 and points to this runner; its old `db/schema.sql` snapshot had drifted beyond the numbered history.

3. Existing databases and failure recovery

1. Back up the database and record the table, column, index, trigger, and view definitions. Confirm which migrations ran, including the two monitoring files numbered 016.
2. Compare the actual schema with the full ordered file set. Resolve any drift through a reviewed forward migration. Only then prepare an explicit adoption record with matching immutable file checksums.
3. If the runner reports `applying`, inspect which statements in that file committed. Repair or complete the partial DDL, then record the verified result. Do not delete a ledger row merely to retry the file.
4. Re-run `--check` before applying pending files. Stop if the ledger checksum differs from the checked-out file.

4. Evidence and limits

Observed: Disposable local MariaDB 11.8 databases accepted all 21 monitoring and 6 SoR files. Re-running checks produced no pending entries. An untracked populated database and a simulated interrupted ledger entry were both refused. The disposable databases were dropped after testing. Xenon has both 016 feature tables and lacks the candidate 020 queue table; this is a read-only observation, not a complete migration audit.

Unverified: Production adoption, restore/rollback, a staged upgrade, live SoR compatibility, and CI execution remain release gates. This candidate runner does not automatically adopt legacy schemas.

Standalone HTML manual with inline CSS and SVG. The matching PDF is generated from this file. No external assets or scripts are required.